

SIMULADOR DE PATOS

Aplicação de Design Patterns no Simulador de Patos



INTEGRANTES

Alex Mendes

.....

Gabriel Madureira

.....

Wendell Leonardo

.....

Yalle Ramos

Tópicos a serem tratados

Objetivo

.....

Reflexões

.....

Porque usamos o Strategy?

Perguntas para análise

1

Qual é a função do padrão escolhido?

2

Como ele pode ser aplicado ao simulador de patos?

3

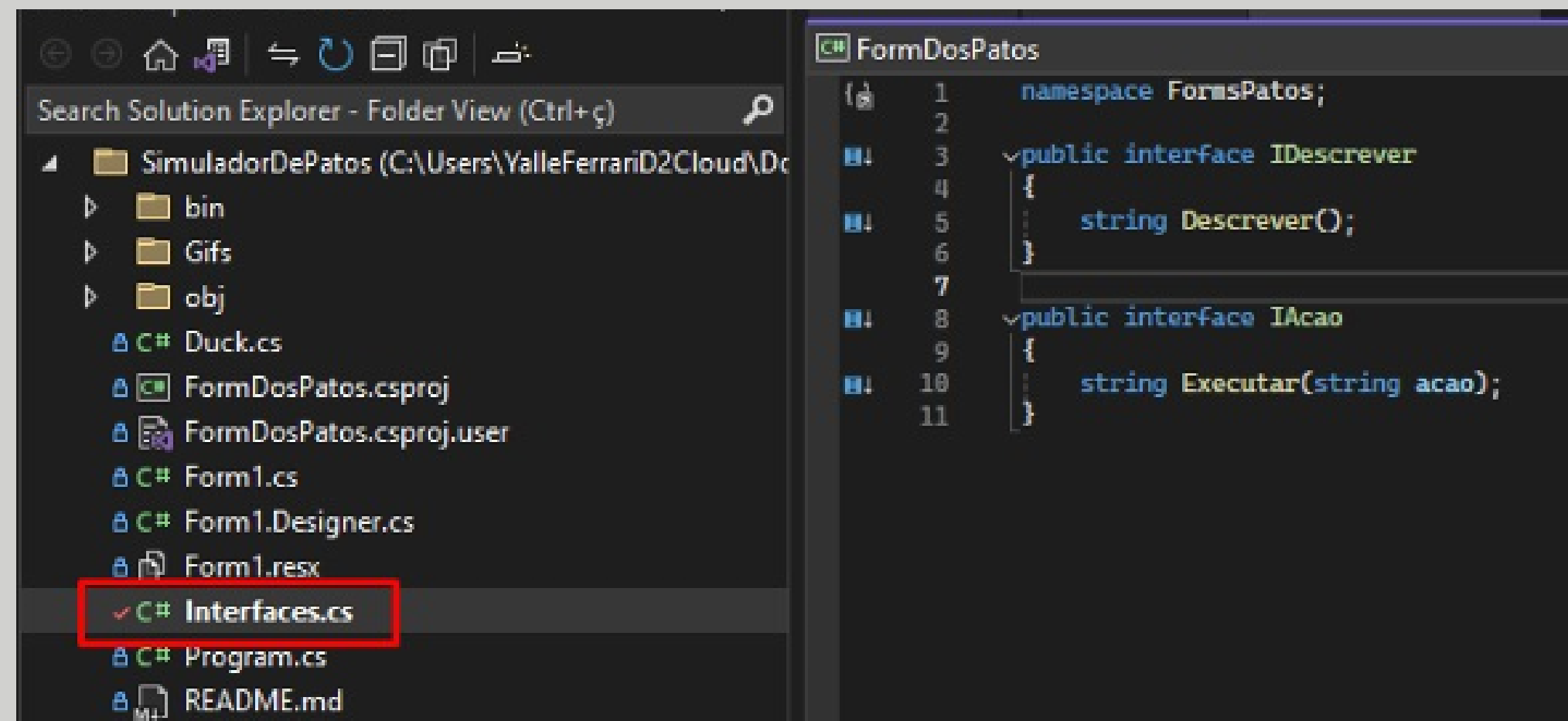
Quais classes precisarão ser modificadas ou adicionadas?

Antes e Depois das classes



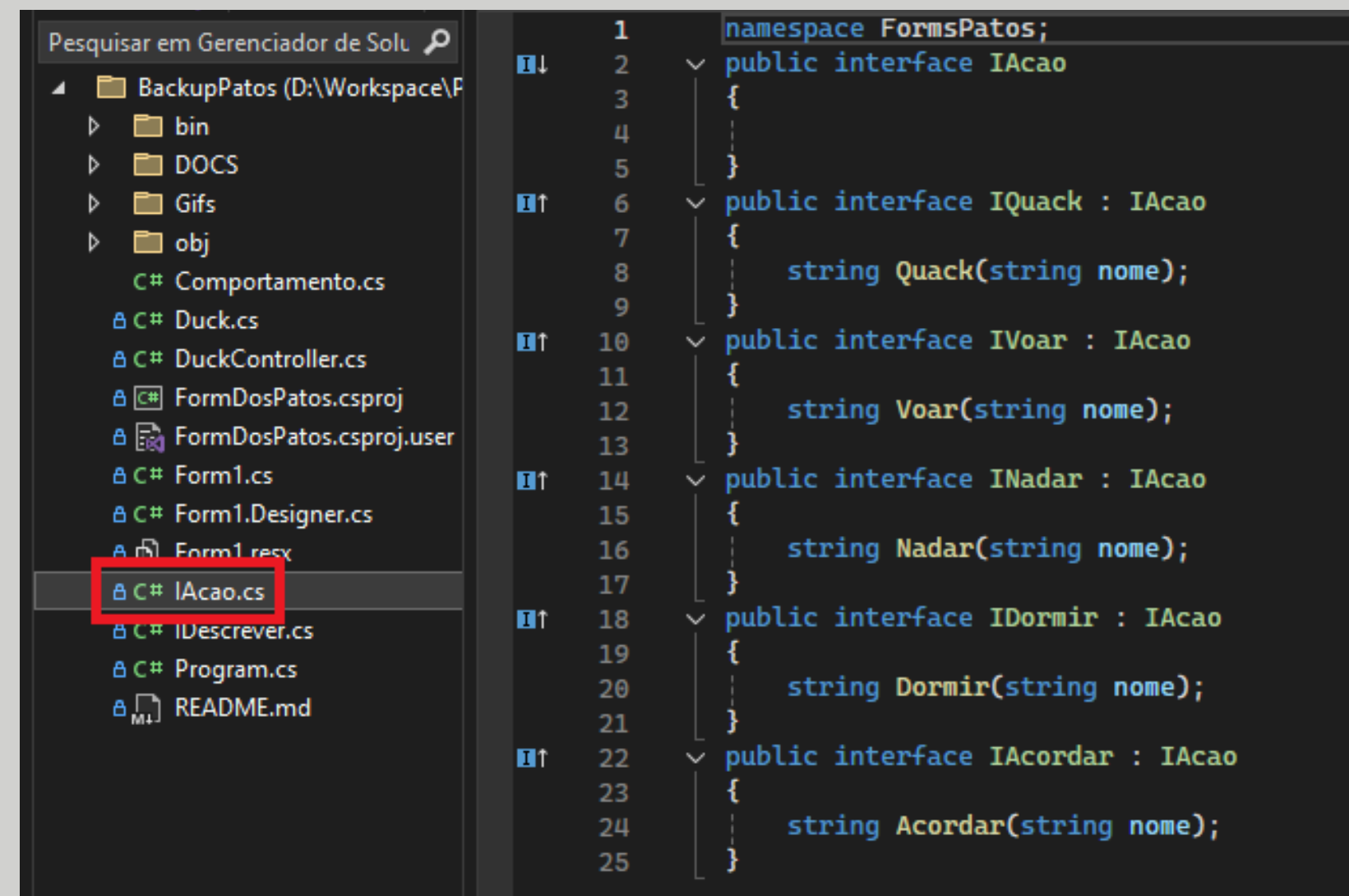
Antes

Classe Interfaces.cs

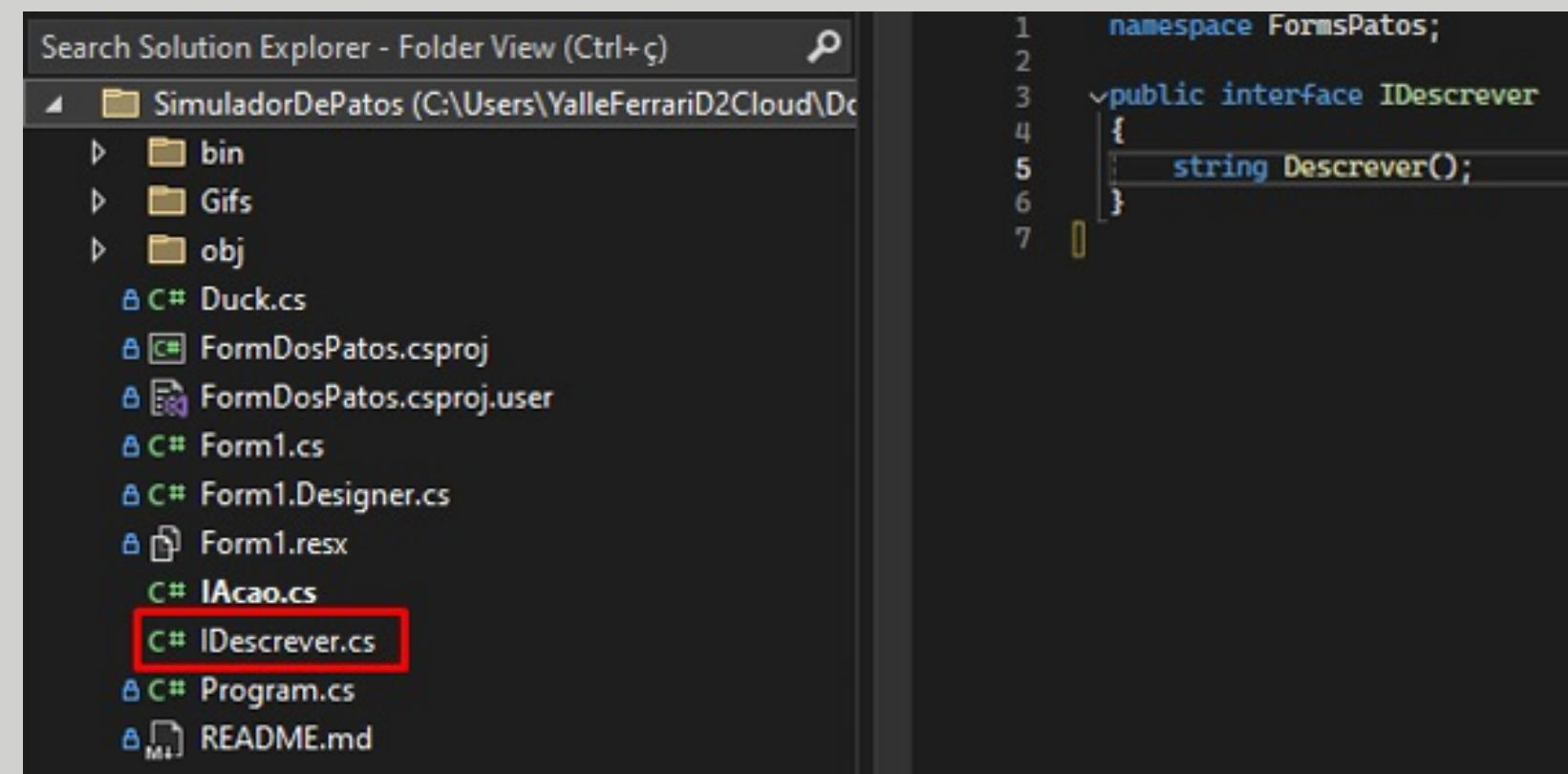


Depois

Classe Interfaces.cs



```
1 namespace FormsPatos;
2 public interface IAcao
3 {
4     ...
5 }
6 public interface IQuack : IAcao
7 {
8     string Quack(string nome);
9 }
10 public interface IVoar : IAcao
11 {
12     string Voar(string nome);
13 }
14 public interface INadar : IAcao
15 {
16     string Nadar(string nome);
17 }
18 public interface IDormir : IAcao
19 {
20     string Dormir(string nome);
21 }
22 public interface IAcordar : IAcao
23 {
24     string Acordar(string nome);
25 }
```



```
1 namespace FormsPatos;
2
3 public interface IDescrever
4 {
5     string Descrever();
6 }
7
```

Antes

Classe Duck.cs

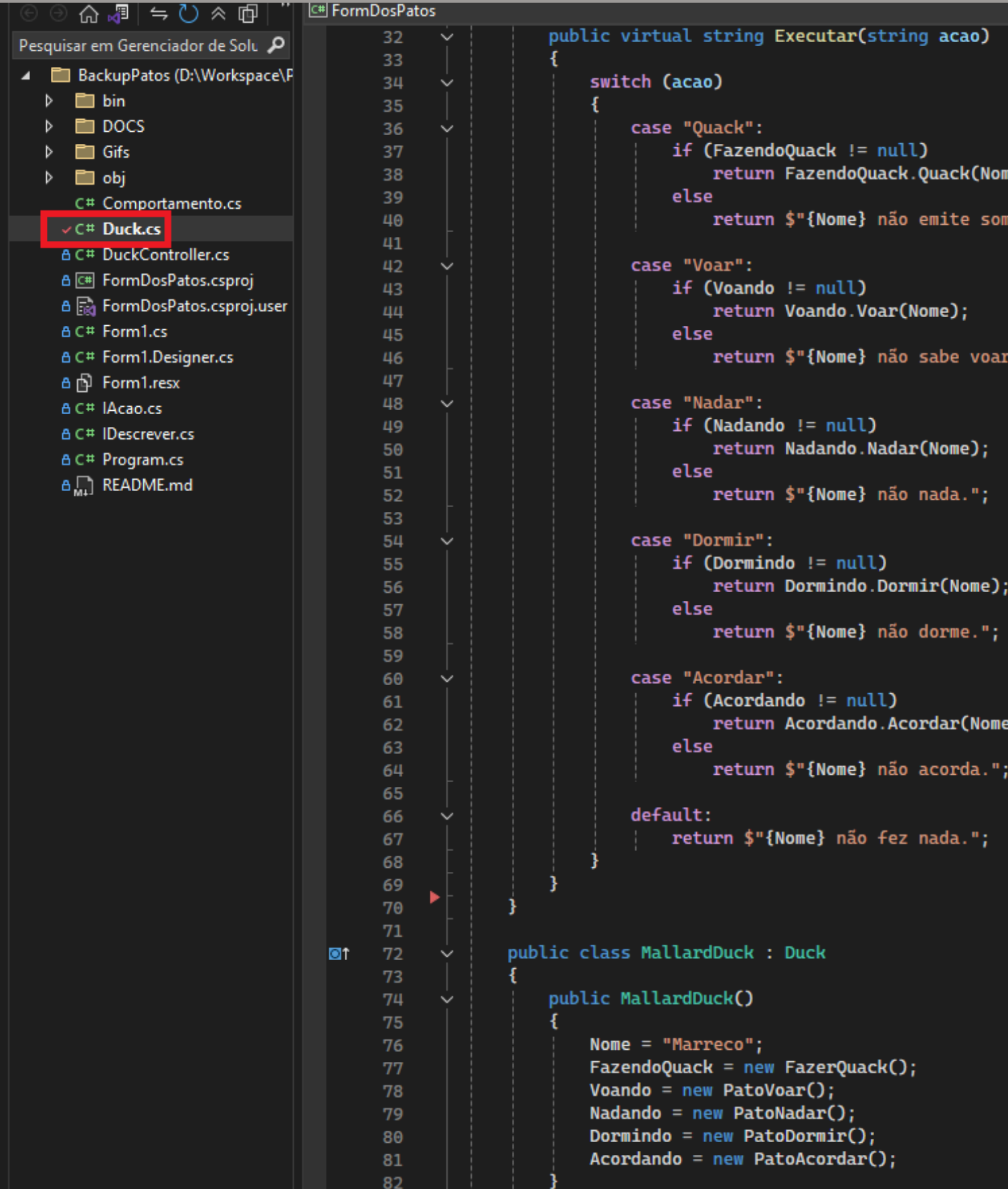
```
public virtual string Executar(string acao)
{
    string resultado;

    switch (acao)
    {
        case "Quack":
            resultado = Nome + " está fazendo quack!";
            break;
        case "Voar":
            resultado = Nome + " está voando!";
            break;
        case "Nadar":
            resultado = Nome + " está nadando!";
            break;
        case "Dormir":
            resultado = Nome + " está dormindo!";
            break;
        case "Acordar":
            resultado = Nome + " acordou!";
            break;
        default:
            resultado = Nome + " não fez nada.";
            break;
    }

    return resultado;
}
```


Depois

Classe Duck.cs



```
32 public virtual string Executar(string acao)
33 {
34     switch (acao)
35     {
36         case "Quack":
37             if (FazendoQuack != null)
38                 return FazendoQuack.Quack(Nome);
39             else
40                 return $"{Nome} não emite som";
41
42         case "Voar":
43             if (Voando != null)
44                 return Voando.Voar(Nome);
45             else
46                 return $"{Nome} não sabe voar";
47
48         case "Nadar":
49             if (Nadando != null)
50                 return Nadando.Nadar(Nome);
51             else
52                 return $"{Nome} não nada.";
53
54         case "Dormir":
55             if (Dormindo != null)
56                 return Dormindo.Dormir(Nome);
57             else
58                 return $"{Nome} não dorme.";
59
60         case "Acordar":
61             if (Acordando != null)
62                 return Acordando.Acordar(Nome);
63             else
64                 return $"{Nome} não acorda.";
65
66         default:
67             return $"{Nome} não fez nada.";
68     }
69 }
70
71
72 public class MallardDuck : Duck
73 {
74     public MallardDuck()
75     {
76         Nome = "Marreco";
77         FazendoQuack = new FazerQuack();
78         Voando = new PatoVoar();
79         Nadando = new PatoNadar();
80         Dormindo = new PatoDormir();
81         Acordando = new PatoAcordar();
82     }
```

Antes

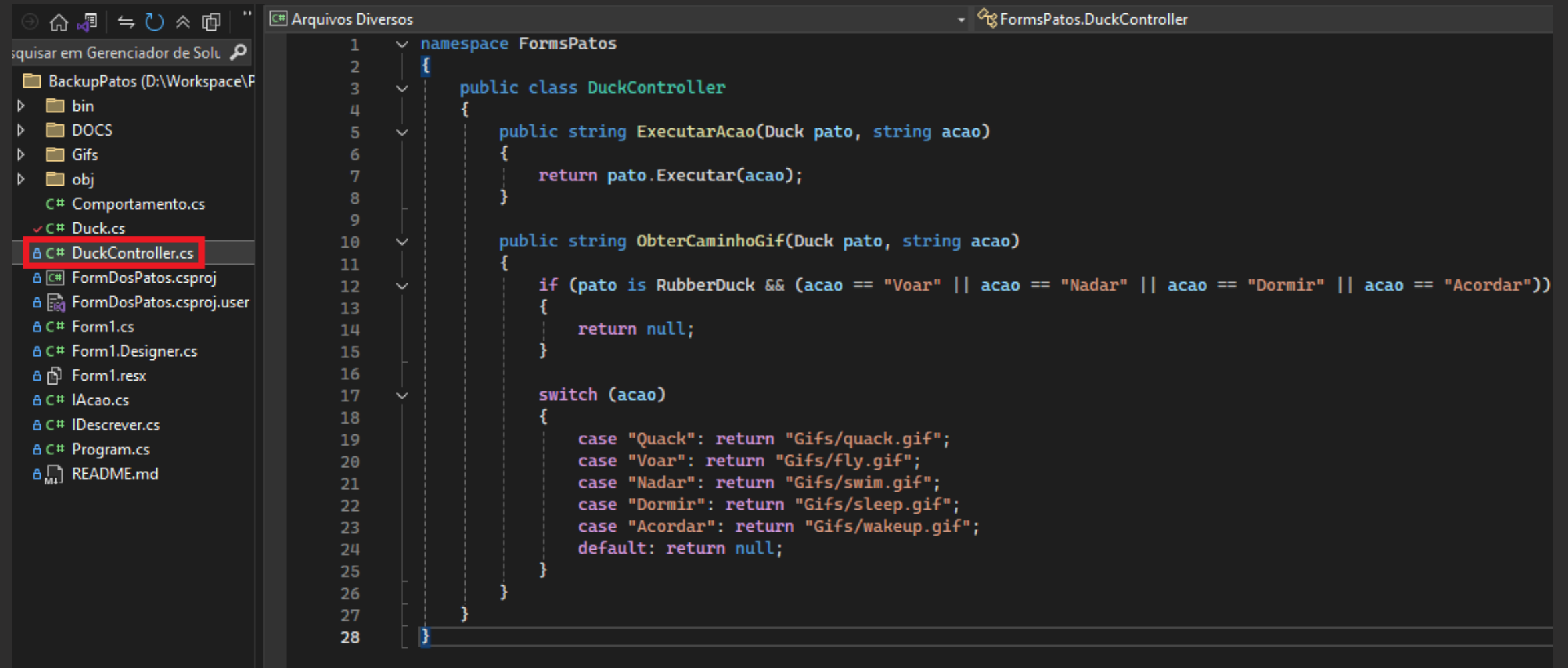
Classe DuckControler.cs

```
public string ObterCaminhoGif(Duck pato, string acao)
{
    if (pato is RubberDuck && (acao == "Voar" || acao == "Dormir" || acao == "Acordar"))
    {
        return null;
    }

    if (acao == "Quack")
        return "Gifs/quack.gif";
    else if (acao == "Voar")
        return "Gifs/fly.gif";
    else if (acao == "Nadar")
        return "Gifs/swim.gif";
    else if (acao == "Dormir")
        return "Gifs/sleep.gif";
    else if (acao == "Acordar")
        return "Gifs/wakeup.gif";
    else
        return null;
}
```


Depois

Classe DuckController.cs

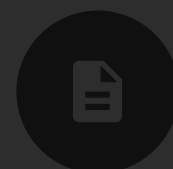


The screenshot shows the Visual Studio IDE. On the left, the Solution Explorer displays a project named 'BackupPatos' with a file tree including 'bin', 'DOCS', 'Gifs', 'obj', 'Comportamento.cs', 'Duck.cs', 'DuckController.cs' (highlighted with a red box), 'FormDosPatos.csproj', 'FormDosPatos.csproj.user', 'Form1.cs', 'Form1.Designer.cs', 'Form1.resx', 'IAcao.cs', 'IDescrever.cs', 'Program.cs', and 'README.md'. The main editor window shows the code for 'FormsPatos.DuckController'. The code is as follows:

```
1 namespace FormsPatos
2 {
3     public class DuckController
4     {
5         public string ExecutarAcao(Duck pato, string acao)
6         {
7             return pato.Executar(acao);
8         }
9
10        public string ObterCaminhoGif(Duck pato, string acao)
11        {
12            if (pato is RubberDuck && (acao == "Voar" || acao == "Nadar" || acao == "Dormir" || acao == "Acordar"))
13            {
14                return null;
15            }
16
17            switch (acao)
18            {
19                case "Quack": return "Gifs/quack.gif";
20                case "Voar": return "Gifs/fly.gif";
21                case "Nadar": return "Gifs/swim.gif";
22                case "Dormir": return "Gifs/sleep.gif";
23                case "Acordar": return "Gifs/wakeup.gif";
24                default: return null;
25            }
26        }
27    }
28 }
```

Testes e Ajustes

Guias para teste:



O código reflete o padrão corretamente?



A nova funcionalidade funciona dentro do simulador?



O código ficou mais organizado e reutilizável?



OBRIGADO